

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САПР



Практична робота №3

з дисципліни: “Застосування систем штучного інтелекту у технологічних рішеннях”

на тему:

“Формування набору даних для виявлення спаму у листуванні”

Виконав:

Ст. групи ПП-44

Верещак Б. О.

Прийняв:

доц. Мокрицька О. В.

Мета роботи

Отримати практичні навички у формуванні вимог до системи штучного інтелекту (ШІ), зокрема для задачі класифікації тексту. Опанувати методи командного аналізу вимог (мозковий штурм, макетування, побудова контекстних діаграм) для створення якісного набору даних – критичного компоненту для навчання моделі машинного навчання. Сформулювати функціональні вимоги та технічне завдання шляхом переговорів між командами.

Теоретичні відомості

З розвитком електронної пошти та месенджерів, проблема спаму стала глобальною. Спам — це небажані повідомлення, які можуть містити рекламу, шахрайські посилання або шкідливе програмне забезпечення. Для автоматичного виявлення спаму використовують машинне навчання та штучний інтелект, зокрема методи класифікації текстів і нейронні мережі.

Мета підготовки датасету – надати моделі структуровані дані для навчання, що дозволяє підвищити точність класифікації.

1. Набір даних (Dataset)

1) Структура

Для задачі класифікації текстів датасет повинен містити:

- Вхідні дані: текст повідомлення (email, SMS, чат).
- Мітки (labels): “spam” або “not spam” (легітимне повідомлення).

2) Джерела даних

- Відкриті датасети:
 - Enron Spam Dataset (посилання)
 - SMS Spam Collection Dataset (посилання)
- Власні дані: анонімізовані листи з корпоративної пошти або чатів.

2. Передобробка даних

Перед використанням дані потрібно очистити та нормалізувати:

- 1.) Очищення тексту: видалення HTML-тегів, URL, емодзі.
- 2.) Нормалізація: приведення тексту до нижнього регістру.
- 3.) Токенізація: розбиття тексту на слова.
- 4.) Видалення стоп-слів: слова, які не несуть смислового навантаження (“and”, “the”, “i”, “na”).
- 5.) Стемінг або лемматизація: зведення слів до базової форми (“running” → “run”).

3. Представлення тексту

Щоб текст можна було використовувати для моделей машинного навчання, його потрібно перетворити у числовий формат:

- Bag-of-Words (BoW) — просте підрахування частоти слів.
- TF-IDF (Term Frequency–Inverse Document Frequency) — зважене представлення, що враховує важливість слова в контексті всього корпусу.

- Word Embeddings (Word2Vec, GloVe, BERT) — представлення слів у вигляді векторів, що зберігають семантичні відносини

Практичне завдання

2.1. Підготовчий етап

1. Ознайомитися з відкритими наборами даних для виявлення спаму.
2. Визначити формат даних для подальшої роботи (CSV, JSON).

2.2. Основний етап

1. Завантажити обраний датасет.
2. Провести передобробку текстів: очищення, токенизація, нормалізація.
3. Розділити датасет на навчальний та тестовий набори (наприклад, 80%/20%).
4. Застосувати метод представлення тексту (BoW або TF-IDF).
5. Зберегти готовий датасет у вигляді таблиці із двома колонками: text та label.

Програмний код з аналізом:

Для виконання завдання я обрав "SMS Spam Collection Dataset" від UCI. Це класичний, добре збалансований набір даних у форматі CSV (технічно, TSV – з роздільником-табуляцією), що містить колонки "label" (ham/spam) та "text".

Етапи обробки:

1. **Завантаження:** Я використав бібліотеку pandas для завантаження даних. При завантаженні я одразу перейменував колонки на label та text для зручності та вказав encoding='latin-1', оскільки файл містить специфічні символи.

```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
import ssl
# --- 1. Налаштування NLTK ---
setup_nltk()

# --- 2. Завантаження даних ---
url = "/root/spam.csv"
try:
    df = pd.read_csv(url, encoding='latin-1')
    # Обираємо тільки потрібні колонки та перейменовуємо
    df = df[['v1', 'v2']]
    df.columns = ['label', 'text']
    print("Дані успішно завантажено.")
    print(df.head())
except Exception as e:
    print(f"Помилка завантаження даних: {e}")
```

2. **Бінарне кодування міток:** Модель машинного навчання не розуміє текст "ham" або "spam". Тому я перетворив ці мітки у числовий формат: "ham" (легітимне) стало 0, а "spam" – 1.

```
# --- 3. Передобробка міток ---
# Конвертуємо 'ham' -> 0, 'spam' -> 1
df['label'] = df['label'].map({'ham': 0, 'spam': 1})
```

3. **Передобробка тексту:** Це був найважливіший етап. Я створив функцію `preprocess_text`, яка виконує наступні кроки для кожного повідомлення:

- **Очищення (Regex):** Я використав регулярні вирази (`re.sub`) для видалення всіх символів, окрім літер та пробілів. Це ефективно прибирає пунктуацію, цифри та спецсимволи.
- **Нормалізація:** Весь текст був приведений до нижнього регістру (`.lower()`).
- **Токенізація:** Текст був розбитий на окремі слова (токени) за допомогою `word_tokenize` з бібліотеки `nlTK`.
- **Видалення стоп-слів:** Я використав стандартний набір англійських стоп-слів (як-от 'and', 'the', 'is') з `nlTK.corpus.stopwords`, щоб видалити слова, які не несуть смислового навантаження.
- Після очищення я знову "зшив" список токенів назад у єдиний рядок. Це необхідно, оскільки векторизатор (TF-IDF) очікує на вхід саме рядки, а не списки слів.

```
# --- 4. Передобробка тексту ---
print("\nПочаток передобробки тексту...")
# Отримуємо набір англійських стоп-слів
stop_words = set(stopwords.words('english'))

# Застосовуємо функцію передобробки до кожного тексту
df['text'] = df['text'].apply(lambda x: preprocess_text(str(x), stop_words))
print("Передобробку тексту завершено.")
print("Очищений DataFrame (head):")
print(df.head())
```

4. **Розділення даних:** Я розділив весь очищений датасет на тренувальний (80%) та тестовий (20%) набори за допомогою `train_test_split` з `sklearn`. Це критично важливо, щоб оцінити, як модель буде працювати на нових, небачених даних.

```
# --- 5. Розділення датасету ---
X = df['text']
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42, stratify=y)

print(f"\nРозмір тренувального набору: {len(X_train)}")
print(f"Розмір тестового набору: {len(X_test)}")
```

5. **Представлення тексту (Векторизація):** Я обрав метод **TF-IDF**, оскільки він значно ефективніший за простий Bag-of-Words.

TF-IDF (Term Frequency–Inverse Document Frequency) працює так:

- **TF (Частота терміну):** Підраховує, як часто слово з'являється в одному SMS.
- **IDF (Зворотна частота документа):** Дивиться, наскільки слово рідкісне у всіх SMS. Слова, які є всюди (як "day"), отримують низьку вагу.
- **Результат:** Перемноживши TF та IDF, ми отримуємо вагу слова. Слова, які є частими в одному SMS, але рідкісними у всій базі (наприклад, "free", "winner", "congrats"), отримують високу вагу. Це робить їх ідеальними ознаками для виявлення спаму.

Я навчив `TfidfVectorizer` на тренувальних даних (`X_train`) і потім трансформував ним і `X_train`, і `X_test`.

```
# --- 6. Застосування TF-IDF ---
print("\nЗастосування TF-IDF векторизації...")
tfidf_vectorizer = TfidfVectorizer(max_features=5000) # Обмежимо 5000
найпопулярнішими словами

# Навчаємо векторизатор на тренувальних даних
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
# Трансформуємо тестові дані
X_test_tfidf = tfidf_vectorizer.transform(X_test)

print(f"Форма тренувальної TF-IDF матриці: {X_train_tfidf.shape}")
print(f"Форма тестової TF-IDF матриці: {X_test_tfidf.shape}")
print("Приклад слів зі словника (ознак):")
try:
    print(tfidf_vectorizer.get_feature_names_out()[1000:1010])
except Exception:
    pass
```

6. **Збереження:** Як вимагало завдання, я зберіг готовий до навчання датасет. Я зберіг очищені, але ще не векторизовані, тренувальні та тестові дані у файли `train_dataset.csv` та `test_dataset.csv`. Саме ці файли тепер можна використовувати для навчання будь-якої моделі.

```
# --- 7. Збереження готових датасетів ---
print("\nЗбереження готових датасетів...")
# Створюємо DataFrame з очищеними даними
train_df = pd.DataFrame({'text': X_train, 'label': y_train})
test_df = pd.DataFrame({'text': X_test, 'label': y_test})

# Зберігаємо у CSV
train_df.to_csv('train_dataset.csv', index=False, encoding='utf-8')
test_df.to_csv('test_dataset.csv', index=False, encoding='utf-8')

print("Файли 'train_dataset.csv' та 'test_dataset.csv' успішно створено.")
```

Результати роботи коду:

Після завантаження даних, `df.head()` показував:

```
Завантаження NLTK даних (stopwords, punkt)...\nNLTK дані завантажено.\nДані успішно завантажено.\n  label                                     text\n0  ham  Go until jurong point, crazy.. Available only ...\n1  ham                                     Ok lar... Joking wif u oni...\n2  spam  Free entry in 2 a wkly comp to win FA Cup fina...\n3  ham  U dun say so early hor... U c already then say...\n4  ham  Nah I don't think he goes to usf, he lives aro...\n[nltk_data] Downloading package stopwords to /root/nltk_data...\n[nltk_data]   Package stopwords is already up-to-date!\n[nltk_data] Downloading package punkt to /root/nltk_data...\n[nltk_data]   Package punkt is already up-to-date!
```

Після запуску функції `preprocess_text`, приклад тексту `Free entry in 2 a wkly comp to win FA Cup...` перетворився на `free entry wkly comp win fa cup ....`

```
Початок передобробки тексту...\nПередобробку тексту завершено.\nОчищений DataFrame (head):\n  label                                     text\n0      0  go jurong point crazy available bugis n great ...\n1      0                                     ok lar joking wif u oni\n2      1  free entry wkly comp win fa cup final tkts st ...\n3      0                                     u dun say early hor u c already say\n4      0  nah dont think goes usf lives around though
```

Розділення даних дало наступні розміри:

- `X_train` (тренувальні тексти): 4457 зразків
- `X_test` (тестові тексти): 1115 зразків
- `y_train` (тренувальні мітки): 4457 зразків
- `y_test` (тестові мітки): 1115 зразків

```
Розмір тренувального набору: 4457\nРозмір тестового набору: 1115
```

Після застосування `TfidfVectorizer` (який виявив 6094 унікальних слів у тренувальному наборі):

- Матриця `X_train_tfidf` отримала форму: (4457, 6094)
- Матриця `X_test_tfidf` отримала форму: (1115, 6094)

```
Застосування TF-IDF векторизації...\nФорма тренувальної TF-IDF матриці: (4457, 5000)\nФорма тестової TF-IDF матриці: (1115, 5000)\nПриклад слів зі словника (ознак):\n['fantasies' 'fantastic' 'fantasy' 'far' 'farm' 'farrell' 'fassyole'\n 'fast' 'faster' 'fastest']
```

Це означає, що кожне SMS тепер представлене як вектор з 6094 чисел, де кожне число – це TF-IDF вага конкретного слова.

На фінальному етапі було успішно створено два файли:

- `train_dataset.csv` (4457 рядків)

- test_dataset.csv (1115 рядків) Кожен файл містить колонки text (з очищеним текстом) та label (з мітками 0 або 1).

Збереження готових датасетів...

Файли 'train_dataset.csv' та 'test_dataset.csv' успішно створено.
Тренувальний датасет

	text	label
184	going nothing greatbye	0
2171	wont wats wit guys	0
5422	ok ksry knw sivatats askd	0
4113	stand away doesnt heart ache without dont wond...	0
4588	finished work yet something	0
...	...	...
1932	jus finished avatar nigro	0
5316	jus finish watching tv u	0
2308	moby pub quizwin high street prize u know new ...	1
1903	free entry weekly comp chance win ipod txt pod...	1
763	nothing jus tot u would ask cos u ba gua went ...	0

[4457 rows x 2 columns]

Тестовий датасет

	text	label
2826	oh right ok ill make sure loads work day got r...	0
3695	tirupur call da	0
3906	means fat head	0
575	cash prize claim call	1
2899	come aftr ltdecimalgt cleaning house	0
...	...	...
854	stop story ive told ive returned hes saying order	0
5044	sent jd customer service cum accounts executiv...	0
2015	read shame tell takes runs blame u ever really...	0
3381	well meant opposed drunken night	0
785	supposed couldnt make shes still town though	0

[1115 rows x 2 columns]

Висновки

Під час виконання цієї лабораторної роботи я пройшов повний цикл підготовки текстового набору даних для задачі машинного навчання. Я успішно завантажив та проаналізував сирий "SMS Spam Collection Dataset".

Я опанував ключові етапи передобробки тексту: очищення від "шуму" за допомогою регулярних виразів, нормалізацію (приведення до нижнього регістру) та токенизацію. Особливо корисним було видалення стоп-слів за допомогою бібліотеки nltk, що дозволило зменшити розмірність даних та сфокусувати модель на значущих словах.

Найважливішим кроком було перетворення очищеного тексту у числовий формат за допомогою методу TF-IDF. Я зрозумів, чому цей метод є набагато потужнішим за простий підрахунок слів (BoW), оскільки він надає більшої ваги словам, які є одночасно інформативними для одного документа та рідкісними для всього корпусу.

Я успішно розділив дані на тренувальний та тестовий набори, що є обов'язковою практикою для об'єктивної оцінки моделі, та зберіг фінальні, готові до використання датасети у форматі CSV.

В результаті я отримав не просто набір даних, а якісний, очищений та структурований *актив*, повністю готовий для навчання будь-якої моделі класифікації – від логістичної регресії до нейронних мереж.